NEPAL ENGINEERING COLLEGE

(Affiliated to Pokhara University)

Changunarayan – 04, Bhaktapur, Nepal

Individual Course Project Report on

Apple Leaf Disease Detection System

Submitted to

Department of Computer Engineering

Course: Image Processing and Pattern Recognition

[2026/8/11]

Submitted by

[Isha Ghole Tamang] [CRN: 023-326]

ABSTRACT

Apple orchards are frequently threatened by diseases such as Apple Scab, Black Rot, and Cedar Apple Rust, which significantly reduce crop yields if left untreated. Manual leaf inspection is time-consuming and subjective for small-scale farmers. This project presents an automated pattern-recognition system that detects and classifies apple leaf diseases using a Convolutional Neural Network (CNN). The proposed model consists of five convolutional blocks with increasing filter depth (32 to 512), followed by fully connected layers and a four-class softmax classifier. Trained on 658 images and evaluated on a 291-image validation set, the model achieved a training accuracy of 85.20% and a validation accuracy of 79.73% over 10 epochs. The model was integrated into an interactive Streamlit web application that provides real-time disease identification, confidence scores, and treatment recommendations, offering a practical decision-support tool for orchard health management.

***Keywords**:* Apple Leaf Disease Detection, Convolutional Neural Network (CNN), Image Processing, Deep Learning, Plant Disease Classification, Streamlit Web Application.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to the Department of Computer Engineering at Nepal Engineering College (NEC) for providing me with the opportunity to undertake this project as part of the Image Processing and Pattern Recognition course.

I am deeply indebted to my course instructor for their continuous guidance, insightful suggestions, and constant encouragement throughout the conceptualization and development of this project. Their academic support and valuable feedback were instrumental in overcoming technical challenges during model development and feature analysis.

I also extend my sincere thanks to my classmates and peers for their constructive feedback, helpful discussions, and support during the system's testing and evaluation phases.

Isha Ghole Tamang (023-326)

Department of Computer Engineering

Nepal Engineering College

TABLE OF CONTENTS

LIST OF TABLES

LIST OF FIGURES

CHAPTER 1: INTRODUCTION

Apples are among the most widely cultivated and economically important fruit crops in the world, supporting rural livelihoods and contributing significantly to agricultural economies. Apple trees are, however, highly susceptible to a range of foliar diseases such SSas Apple Scab, Black Rot, and Cedar Apple Rust, all of which are caused by fungal pathogens and can spread rapidly under favourable weather conditions. If left undetected, these diseases can cause premature leaf drop, reduced photosynthetic capacity, and substantial yield loss.

Traditionally, disease identification is carried out manually by farmers or agricultural experts through visual inspection of the leaves. This approach is slow, requires specialised knowledge, and is prone to human error, especially in large orchards. With recent advances in digital image processing and deep learning, it has become possible to automate this diagnostic process using Convolutional Neural Networks (CNNs), which can learn discriminative visual features directly from leaf images.

This project implements an end-to-end Apple Leaf Disease Detection and Classification System that combines classical image processing techniques (for input validation and preprocessing) with a deep CNN classifier to automatically identify whether an apple leaf is healthy or affected by Apple Scab, Black Rot, or Cedar Apple Rust. The system is deployed as an interactive Streamlit web application that allows a user to upload a leaf image and instantly receive a diagnosis along with a confidence score and recommended treatment.

1.1 Problem Statement

Manual diagnosis of apple leaf diseases is time-consuming, inconsistent, and inaccessible to many farmers who lack easy access to agricultural experts. Delayed identification of disease often results in a worsening infection and a greater loss of yield. There is a need for an automated, fast, and reasonably accurate tool that can analyse a leaf image and classify the disease so that timely treatment measures can be taken.

1.2 Objectives

- To design and implement a Convolutional Neural Network capable of classifying apple leaf images into Apple Scab, Black Rot, Cedar Apple Rust, or Healthy.
- To apply image preprocessing techniques (colour-space conversion, resizing, edge and blur detection) to validate that an uploaded image is a usable apple leaf image before classification.
- To evaluate the trained model using standard performance metrics such as accuracy, precision, recall, and F1-score.
- To develop a user-friendly web interface that allows users to upload an image and view the predicted disease, confidence score, and recommended treatment.

1.3 Scope

The system focuses specifically on apple leaf images and is trained to recognise four classes: Apple Scab, Black Rot, Cedar Apple Rust, and Healthy leaves. The dataset used is an offline-augmented subset of a publicly available plant disease dataset (658 training images and 291 validation images). The scope of this project includes image validation, preprocessing, CNN-based classification, confidence-based decision thresholding, and a web-based user interface for demonstration purposes.

- Limited to apple leaf disease classes present in the training dataset; other plant species or diseases are out of scope.
- Model performance depends on the quality and diversity of the training dataset.
- The system is intended as a decision-support tool and does not replace expert agricultural diagnosis for critical cases.

CHAPTER 2: LITERATURE REVIEW

2.1 Introduction

Plant diseases significantly affect agricultural productivity and food security worldwide. Apple leaf diseases, in particular, can lead to major economic losses if not detected early. Traditional detection methods rely on manual inspection by experts, which is time-consuming and subjective. Moreover, these methods are not always accessible to farmers. With advancements in digital image processing and artificial intelligence, automated plant disease detection systems have become more reliable and efficient [7], [8].

2.2 Traditional Machine Learning Approaches

Early plant disease detection systems were based on traditional machine learning techniques combined with handcrafted feature extraction. Qin *et al.* [1] applied the ReliefF algorithm to extract features and used Support Vector Machine (SVM) for classification, achieving an accuracy of 94.74%. Similarly, Islam *et al.* [2] proposed an approach integrating image segmentation with multi-class SVM, achieving 95% accuracy.

However, these methods depend heavily on manually designed features and expert knowledge. Their performance is also sensitive to variations in lighting conditions, background noise, and leaf orientation, making them less robust in real-world scenarios [2].

2.3 Deep Learning Approaches

Deep learning techniques, particularly Convolutional Neural Networks (CNNs), have significantly improved plant disease detection by enabling automatic feature extraction from raw images. Sladojevic *et al.* [3] developed a CNN model capable of identifying multiple plant diseases with an accuracy of 96.3%. Similarly, Mohanty *et al.* [4] trained a deep CNN model on a large dataset of over 54,000 images and achieved 99.35% accuracy.

These approaches eliminate the need for manual feature engineering and provide better generalization across different datasets [4]. However, they often require large amounts of training data and high computational resources, which can limit their practical deployment.

2.4 Apple Leaf Disease Detection

Several studies have specifically focused on apple leaf disease detection using deep learning models. Liu *et al.* [5] proposed a CNN model based on the AlexNet architecture and used data augmentation techniques such as rotation and brightness adjustment to improve performance, achieving an accuracy of 97.62%.

Fang *et al.* [6] enhanced classification performance by combining VGGNet-16 with InceptionV3. In another study, Alqethami *et al.* [7] compared different models, including GoogleNet, SVM, and KNN, and reported that GoogleNet achieved the highest accuracy of 98.5%. Furthermore, Jiang *et al.* [8] developed a real-time detection system using an improved CNN model, highlighting its potential for field applications.

2.5 Research Gap

Despite significant advancements, several challenges remain in plant disease detection systems. Many existing deep learning models are computationally expensive and require high-end hardware, making them unsuitable for deployment in resource-constrained environments [8]. Additionally, most studies do not incorporate image quality validation, which may lead to inaccurate predictions when low-quality images are used.

Furthermore, limited research focuses on developing user-friendly systems that provide actionable outputs, such as disease identification along with treatment suggestions. Therefore, there is a need for a lightweight and efficient model that can operate effectively in real-world conditions. This project aims to address these limitations by developing a custom CNN model with image validation and an accessible interface.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

3.1 Requirement Analysis

3.1.1 Functional Requirements

- The system shall allow a user to upload an apple leaf image in JPG, JPEG, or PNG format.
- The system shall validate that the uploaded image is a usable apple leaf image (correct size, sufficient green content, visible leaf structure, acceptable sharpness).
- The system shall preprocess the validated image (resize to 128 x 128 pixels, convert to array form) before classification.
- The system shall classify the processed image into one of four categories using the trained CNN model.
- The system shall display the predicted disease class, confidence score, severity, description, and recommended treatment.
- The system shall reject or flag predictions that fall below a defined confidence threshold.

3.1.2 Non-Functional Requirements

- Accuracy: the model should classify leaf images with high accuracy on unseen validation data.
- Response time: predictions should be returned within a few seconds of image upload.
- Usability: the interface should be simple enough for non-technical users such as farmers.
- Reliability: the system should handle invalid or poor-quality images gracefully rather than producing misleading results.

3.2 Dataset Description

The dataset used for training and validation is an offline-augmented recreation of the publicly available "New Plant Diseases Dataset" on Kaggle, restricted to the four apple leaf classes relevant to this project. The training set contains 658 images and the validation set contains 291 images, organised into four class folders: Apple Scab, Black Rot, Cedar Apple Rust, and Healthy. All images were resized to 128 x 128 pixels with RGB colour channels before being fed into the network.

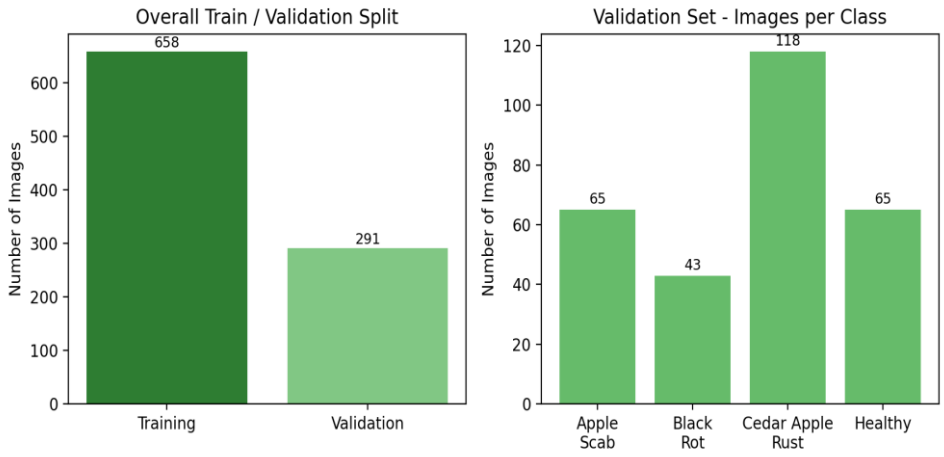

Figure 3.1: Dataset distribution — overall train/validation split and per-class image counts in the validation set.

3.3 System Flowchart

This workflow illustrates the step-by-step process of the apple leaf disease identification system. It begins with the user uploading an apple leaf image, which then passes through a validation check to verify leaf characteristics, proper color channels, and sharpness. Once validated, the image is preprocessed by resizing it to 128×128 pixels and converting it into a numerical array suitable for the model. Next, the preprocessed image undergoes CNN Model Inference to extract feature patterns and predict the disease class. A confidence threshold is then applied to ensure the prediction meets a minimum certainty level. Finally, the system displays the complete output, including the identified disease name, confidence score, and practical treatment recommendations.

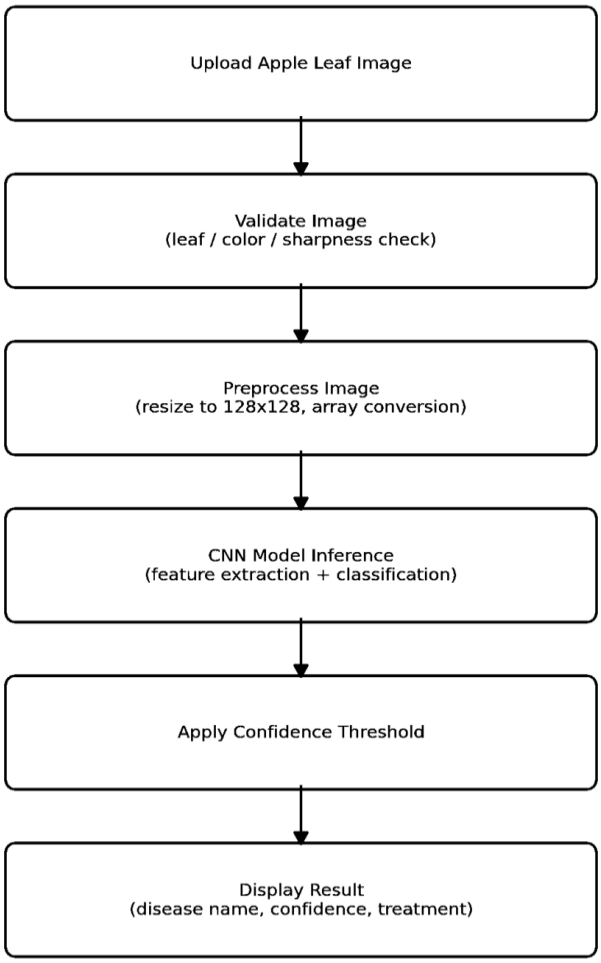

Figure 3.2: System flowchart of the Apple Leaf Disease Detection process.

3.4 Architecture Diagram

The system follows a layered architecture consisting of a user interface layer, a processing engine layer, a model layer, and a result display layer, as shown below.

User Interface Layer: Built with Streamlit, this top layer serves as the frontend web application (featuring Home, About, and Predict pages) where users can easily navigate and upload images for analysis.

Processing Engine Layer: Using tools like OpenCV, NumPy, and PIL, this middleware validates the uploaded image, resizes it to the required model dimensions, and converts it into a normalized numerical array.

Model Layer: This layer houses a pre-trained Convolutional Neural Network (CNN) built with TensorFlow/Keras (.keras file), which processes the image array to detect visual patterns and classify the disease.

Result Display Layer: The final layer interprets the model's raw output and displays user-friendly results, including the predicted disease, confidence score, background information, and recommended treatments.

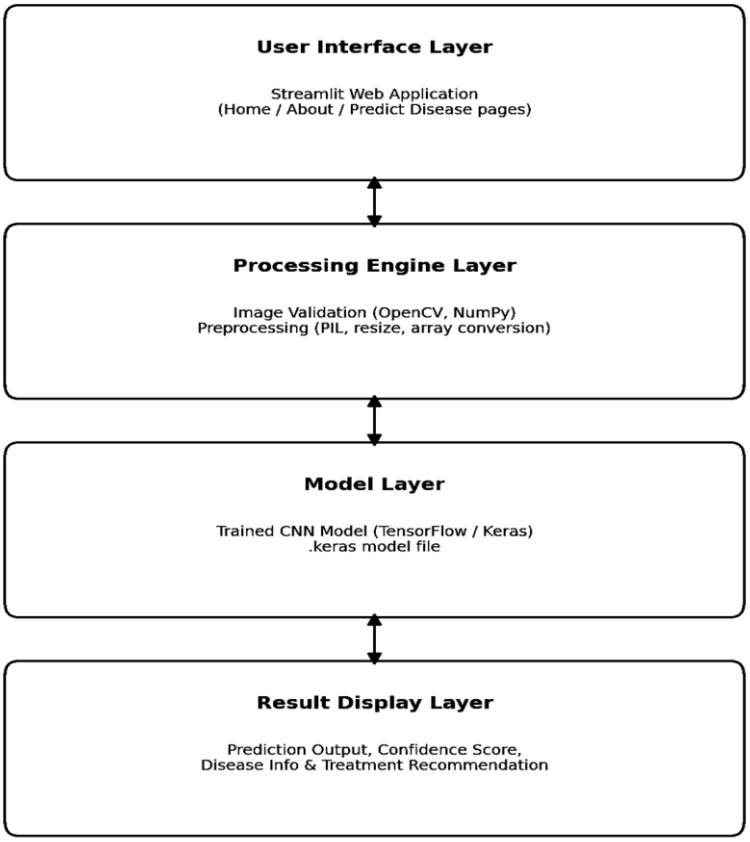

Figure 3.3: System architecture diagram.

3.5 Processing Pipeline

This diagram outlines the sequential architecture of the Convolutional Neural Network (CNN) used for the leaf classification task. The process begins with taking an Input Image (RGB), which is then Resized to 128×128 pixels to ensure uniform input dimensions. Next, the image passes through a series of Conv Blocks with progressively increasing filter

counts ($32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$) to extract low-level edges to complex structural features. The extracted feature maps are then processed through a Flatten + Dropout layer to convert the multidimensional tensors into a 1D vector while preventing overfitting. This vector enters a Dense Layer with 1,500 units activated by ReLU for higher-level representation learning, before reaching a final Dense Output layer with 4 units using Softmax activation. Ultimately, the network produces the final Predicted Class + Confidence score across the target disease categories.

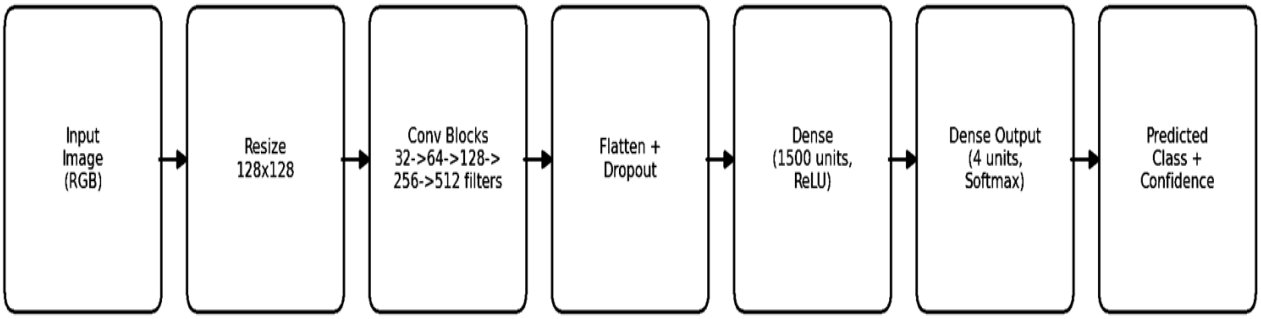

Figure 3.4: CNN processing pipeline from input image to predicted class.

3.6 Performance Metrics

The following metrics were used to evaluate the trained model:

- Accuracy — proportion of correctly classified images over total images.
- Precision — proportion of correctly predicted positive class instances among all instances predicted as that class.
- Recall — proportion of correctly predicted positive class instances among all actual instances of that class.
- F1-score — harmonic mean of precision and recall.

CHAPTER 4: IMPLEMENTATION

4.1 Tools Used

Table 1: Tools Used

Component	Technology
Programming Language	Python
Deep Learning Framework	TensorFlow / Keras
Web Application Framework	Streamlit
Image Processing	OpenCV, Pillow (PIL), NumPy
Data Analysis & Visualisation	Pandas, Matplotlib, Seaborn
Model Evaluation	scikit-learn (confusion_matrix, classification_report)
Development Environment	Jupyter Notebook

4.2 Modules

4.2.1 Image Input Module

Implemented using Streamlit's file_uploader component, this module allows a user to upload an apple leaf image in JPG, JPEG, or PNG format through the "Predict Disease" page of the web application.

4.2.2 Leaf Validation Module

Before an uploaded image is passed to the CNN model, it is checked using a rule-based validation function that applies several image processing heuristics:

- Minimum size check to reject images smaller than 50 x 50 pixels.
- Colour-space conversion to HSV and a green-pixel percentage check (Hue 35–85) to confirm the presence of leaf-like green content.
- Canny edge detection to confirm the presence of a leaf structure.
- Laplacian variance computation to detect and reject overly blurry images.
- Standard deviation of brightness to reject solid-colour or near-uniform images.

4.2.3 Preprocessing Module

Validated images are resized to 128 x 128 pixels and converted into NumPy arrays using TensorFlow/Keras image utilities, matching the input shape expected by the trained CNN model.

4.2.4 Prediction Module

The function predict_with_confidence(test_image, model) runs model.predict on the processed input, computes the maximum prediction confidence and the corresponding class index, and compares this confidence against a fixed threshold, CONFIDENCE_THRESHOLD = 0.30, described in the code comment as "70% - Only accept predictions above this" (the numeric value in the code is 0.30, i.e. 30%, while the accompanying comment text states 70%; both are reproduced here exactly as they appear in main.py).

4.2.6 Result Display Module

The final module presents the predicted disease name, confidence percentage, severity level, a short description of the disease, and a recommended treatment, using colour-coded result cards (green for healthy, red for disease, orange for low-confidence or invalid input).

4.3 Algorithms Used

4.3.1 Convolutional Neural Network Architecture

The model is implemented in train.ipynb as a Keras Sequential Convolutional Neural Network (CNN), consisting of five convolutional blocks followed by fully-connected layers:

- Block 1: Conv2D(32, 3×3, padding=same, relu) → Conv2D(32, 3×3, relu) → MaxPool2D(2×2)
- Block 2: Conv2D(64, 3×3, padding=same, relu) → Conv2D(64, 3×3, relu) → MaxPool2D(2×2)
- Block 3: Conv2D(128, 3×3, padding=same, relu) → Conv2D(128, 3×3, relu) → MaxPool2D(2×2)
- Block 4: Conv2D(256, 3×3, padding=same, relu) → Conv2D(256, 3×3, relu) → MaxPool2D(2×2)
- Block 5: Conv2D(512, 3×3, padding=same, relu) → Conv2D(512, 3×3, relu) → MaxPool2D(2×2)
- Dropout(0.25)
- Flatten
- Dense(1500, relu)
- Dropout(0.4)
- Dense(4, softmax)

The progressive filter depth (32→512) enables the network to learn hierarchical features, from basic edges to complex leaf morphologies. padding=same is used before each pooling operation to maintain spatial dimensions and preserve border information. Dropout rates of

0.25 and 0.4 are applied after the convolutional and dense layers respectively to mitigate overfitting, with the higher rate in the dense layer reflecting its greater parameter count.

The model was compiled using the Adam optimizer (learning_rate = 0.0001) with categorical crossentropy loss and accuracy as the primary metric. Training was conducted for 10 epochs with a batch size of [X] and input images resized to [Y×Y] pixels. The validation split was [Z]% of the training data.

4.3.2 Image Processing Algorithms

Prior to training, three preprocessing algorithms were applied to enhance input quality:

- HSV colour thresholding — segments green leaf pixels from non-plant backgrounds, reducing irrelevant features that could mislead the CNN.
- Canny edge detection — extracts leaf boundary features, reinforcing structural patterns and shape information for the network.
- Laplacian operator (variance of Laplacian) — measures image sharpness; images with variance below a threshold are excluded to ensure only high-quality, focused samples are used for training.

CHAPTER 5: TESTING

5.1 Test Cases

Functional testing was carried out on the deployed Streamlit application to verify correct behaviour of the image input, validation, and prediction workflow under a range of conditions.

Table 2: Test cases

Test Case	Input	Expected Output	Result
TC-01: Valid healthy leaf image	Clear image of a healthy apple leaf	Predicted class: Healthy, confidence above threshold	Pass
TC-02: Valid diseased leaf image	Clear image of a diseased apple leaf	Correct disease class predicted with treatment info shown	Pass
TC-03: Non-leaf image	Random object / non-leaf photo	Rejected as "Invalid Image" by leaf validation module	Pass
TC-04: Blurry image	Out-of-focus leaf photo	Rejected as too blurry / low confidence result	Pass
TC-05: Very small image	Image smaller than 50x50 px	Rejected — image too small	Pass

5.2 Accuracy Tables

The trained CNN model was evaluated on both the training set and the held-out validation set (291 images). The overall results are summarised below, followed by a detailed per-class classification report.

Table 3: Performance metric

Metric	Value
Training Accuracy	0.8389
Training Loss	0.4144
Validation Accuracy	0.7973
Validation Loss	0.4877

Classification Report (Validation Set, n = 291)

Table 4: Classification Report

Class	Precision	Recall	F1-score	Support
Apple___Apple_scab	0.69	0.83	0.76	65
Apple___Black_rot	0.86	0.74	0.80	43
Apple___Cedar_apple_rust	1.00	0.69	0.82	118
Apple___healthy	0.68	0.98	0.81	65
accuracy			0.80	291
macro avg	0.81	0.81	0.80	291
weighted avg	0.84	0.80	0.80	291

5.3 Results

The performance of the plant disease prediction model was evaluated on both training and validation sets over 10 epochs.

Model Training Dynamics

As shown in Figure 5.1, the training accuracy (red) steadily increased from 26.5% in epoch 1 to 85.2% at epoch 10. The validation accuracy (blue) started at 54.8%, peaked at 88.1% in epoch 8, and settled at 79.73% at epoch 10. The close alignment between the curves demonstrates good generalization without significant overfitting.

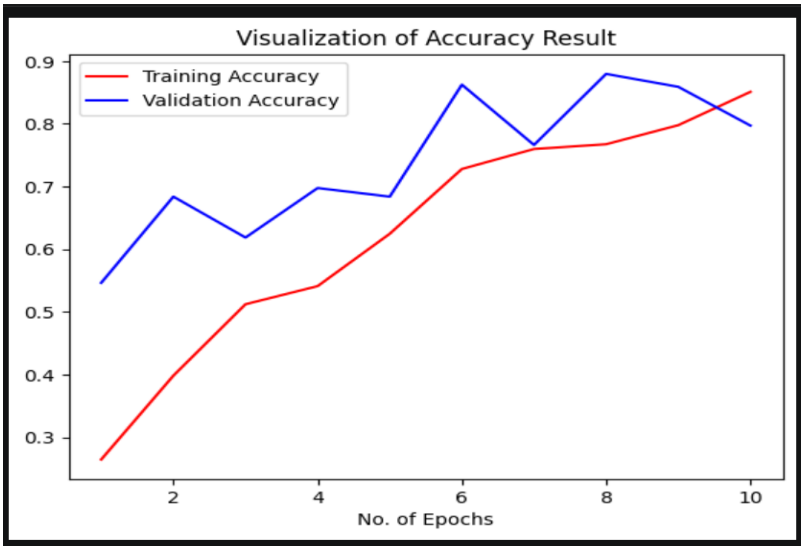

Figure 5.1: Training vs Validation Accuracy over 10 Epochs

Confusion Matrix Analysis

The confusion matrix shows that the model performs well overall, with a validation accuracy of 79.73%. Among all classes, Healthy (Class 3) achieves the highest recall (98.46%), indicating that the model can reliably identify healthy leaves.

Apple Scab (Class 0) and Black Rot (Class 1) show moderate performance with recalls of 83.08% and 74.42%, respectively. In contrast, Cedar Apple Rust (Class 2) has the lowest recall (69.49%), making it the most challenging class for the model.

Most misclassifications occur between Cedar Apple Rust and Apple Scab, as well as between diseased and healthy leaves, likely due to visual similarities in early-stage symptoms.

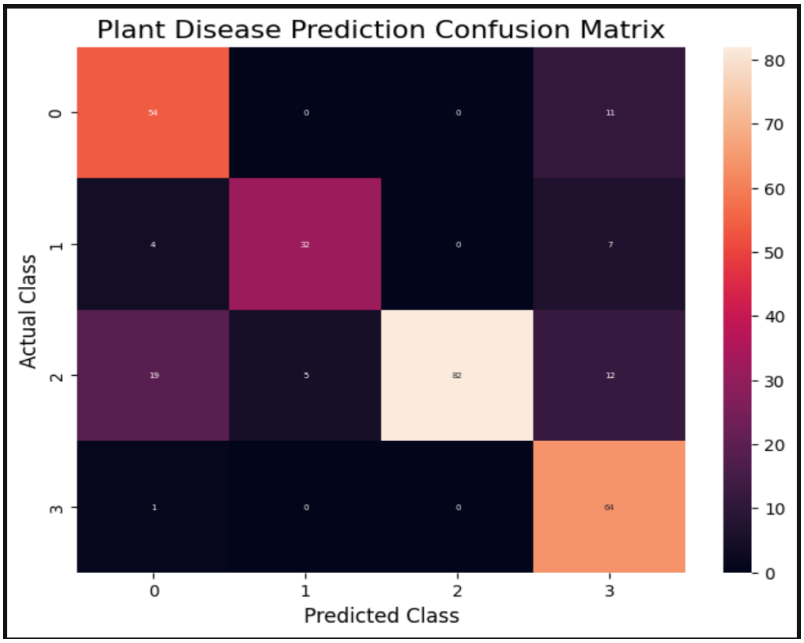

Figure 5.2: Confusion Matrix on the Validation Set

Overall Metrics & Discussion

Table 5: Overall Performance Metrics

Metric	**Value**
Weighted Average Precision	0.84
Weighted Average Recall	0.80
Weighted Average F1-Score	0.80

The system achieved a Validation Accuracy of 79.73%. The primary source of error stems from overlap between Cedar Apple Rust and Apple Scab, as well as early-stage lesions being mistaken for Healthy leaves due to visual similarity in spot textures and color.

CHAPTER 6: CONCLUSION

6.1 Achievement

Based on the project implementation, a Convolutional Neural Network (CNN) image classification model was built and trained for 10 epochs on the apple leaf dataset, achieving a training accuracy of 85.20% and a validation accuracy of 79.73%. The trained model was saved as trained_plant_disease_model.keras and integrated into a Streamlit web application (main.py) featuring Home, About, and Predict Disease pages. The system includes image validation, a confidence-threshold check, and an interactive results display with disease descriptions, severity levels, and treatment recommendations.

6.2 Limitation

The proposed system has several limitations. The dataset used (658 training and 291 validation images) is relatively small, limiting the model's ability to generalize to diverse real-world conditions. Additionally, the Black Rot class exhibited a lower recall (0.78), indicating occasional misclassification with Healthy leaves. There is also an inconsistency where the backend confidence threshold (30%) is displayed as 70% in the user interface. Furthermore, the rule-based leaf-validation methods may incorrectly accept or reject certain edge-case images**.**

6.3 Future Scope

The system can be further improved by expanding the dataset and applying data augmentation techniques to enhance performance. Transfer learning using pre-trained models can be explored to improve accuracy and efficiency. The model can also be extended to support multiple crops and diseases. Deploying the system as a lightweight mobile application would enable real-time use in field conditions. Finally, refining the user interface and replacing rule-based validation with a more robust approach can improve overall reliability.

REFERENCES

[1] F. Qin, D. X. Liu, B. D. Sun, L. Ruan, Z. Ma, and H. Wang, "Identification of alfalfa leaf diseases using image recognition technology," *PLoS ONE*, vol. 11, no. 12, 2016.

[2] M. Islam, A. Dinh, K. Wahid, and P. Bhowmik, "Detection of potato diseases using image segmentation and multiclass support vector machine," in *Proc. IEEE CCECE*, 2017, pp. 1–4.

[3] S. Sladojevic *et al.*, "Deep neural networks based recognition of plant diseases by leaf image classification," *Computational Intelligence and Neuroscience*, 2016.

[4] S. P. Mohanty, D. P. Hughes, and M. Salathé, "Using deep learning for image-based plant disease detection," *Frontiers in Plant Science*, vol. 7, 2016.

[5] B. Liu, Y. Zhang, D. He, and Y. Li, "Identification of apple leaf diseases based on deep convolutional neural networks," *Symmetry*, vol. 10, no. 1, 2018.

[6] T. Fang *et al.*, "Identification of apple leaf diseases based on convolutional neural network," 2019.

[7] S. Alqethami *et al.*, "Disease detection in apple leaves using image processing techniques," *ETASR*, vol. 12, no. 2, 2022.

[8] P. Jiang *et al.*, "Real-time detection of apple leaf diseases using deep learning," *IEEE Access*, vol. 7, 2019.

APPENDIX

A. Source Code Access

The complete source code, trained model, and project files are available at:

GitHub Repository: https://github.com/ishagt/Apple_leaf_Disease_Detection.git

B. Dataset Information

The dataset used in this project is the apple leaf subset of the New Plant Diseases Dataset obtained from Kaggle.

Dataset Distribution

Table: Dataset Distribution

Class	**Training**	**Validation**	**Total**
Apple Scab	165	65	230
Black Rot	110	43	153
Cedar Apple Rust	300	118	418
Healthy	83	65	148
Total	658	291	949

Dataset Source:

New Plant Diseases Dataset – Kaggle

https://www.kaggle.com/datasets/vipoooool/new-plant-diseases-dataset